Warmup Revision

16 problems

Fast recall cards for the completed warmup package. Read the problem name, picture the hook, then check the sample and bug magnets before opening the Java file. Exported in expanded card mode.

#01 ContainsDuplicate

Easy

Seen set Easy Source LeetCode

Before reading: what data structure tells you a value was already seen?

PROBLEM DESCRIPTION

Given an integer array, return true if any value appears at least twice. Return false only when every value is unique.

INPUT

```
nums = [1, 2, 3, 4, 4]
```

OUTPUT

```
Result: true
```

TIME
O(n)

SPACE
O(n)

DATA STRUCTURES
HashSet

WHEN TO RECOGNIZE IT

When the question asks whether anything repeats and you do not need the repeated positions.

VISUAL HOOK

1. Start with an empty seen set.
2. For each number, check before adding.
3. If already seen, return true immediately.

Imagine dropping every number into a seen bucket. If it is already in the bucket, stop.

CORE MOVE

Scan left to right. If seen already contains the number, return true; otherwise add it and continue.

CODE SKELETON

```
seen = new HashSet()
for n in nums:
  if seen.contains(n): return true
  seen.add(n)
return false
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Return immediately when a duplicate appears.
- A set keeps this linear; sorting is extra work and mutates order.

#02 Pangram

Easy

Alphabet checklist Easy Source LeetCode

Before reading: how do you know all 26 letters appeared?

PROBLEM DESCRIPTION

Given a sentence, check whether it contains every English alphabet letter at least once.

INPUT

```
"This is not a pangram"
"TheQuickBrownFoxJumpsOverTheLazyDog"
```

OUTPUT

```
false
true
```

TIME
O(n)

SPACE
O(1)

DATA STRUCTURES
Boolean[26] checklist

WHEN TO RECOGNIZE IT

When the task is presence of all alphabet letters, not order or frequency.

VISUAL HOOK

a b c ... z

1. Map each letter to a box 0..25.
2. Mark the box when seen.
3. All boxes must be marked.

Picture 26 boxes from a to z. Every seen character checks one box; all boxes must be checked.

CORE MOVE

Normalize each character, map a-z to indexes 0-25, mark/count each letter, then ensure none are missing.

CODE SKELETON

```
seen = boolean[26]
for c in sentence:
  if 'a' <= lower(c) <= 'z': seen[c-'a'] =
true
return all seen values are true
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Ignore characters outside a-z if input can contain spaces or punctuation.
- You can early-return once all 26 letters have been seen.

#03 ReverseVowels

Easy

Vowel swap Easy Source LeetCode

Before reading: which characters are allowed to move?

PROBLEM DESCRIPTION

Given a string, reverse only its vowels. Every consonant, digit, symbol, and space stays in its original position.

INPUT

```
"hello"
"DesignGUrus"
```

OUTPUT

```
holle
DusUgnGires
```

TIME

O(n)

SPACE

O(n)

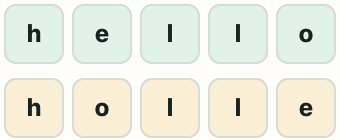
DATA STRUCTURES

char array, two pointers, vowel lookup

WHEN TO RECOGNIZE IT

When only one character class moves and all other characters stay anchored.

VISUAL HOOK



1. Move left until it finds a vowel.
2. Move right until it finds a vowel.
3. Swap vowels and move both inward.

Two magnets search inward for vowels. When both magnets catch vowels, swap them.

CORE MOVE

Use two pointers. Skip non-vowels on each side. When both sides are vowels, swap and move inward.

CODE SKELETON

```
l = 0; r = n - 1
while l < r:
  while l < r && !isVowel(chars[l]): l++
  while l < r && !isVowel(chars[r]): r--
  swap(chars, l++, r--)
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Include uppercase vowels by normalizing before lookup.
- After a swap, move both pointers or you can keep revisiting the same vowels.

#04 ValidPalindrome

Easy

Clean two-pointer compare Easy Source LeetCode

Before reading: what must each pointer skip before comparing?

PROBLEM DESCRIPTION

Given a string, check whether it reads the same forward and backward after ignoring non-alphanumeric characters and case.

INPUT

```
"A man, a plan, a canal, Panama!"
"Was it a car or a cat I saw?"
"123321"
```

OUTPUT

```
true
true
true
```

TIME	SPACE	DATA STRUCTURES
O(n)	O(1)	Two pointers

WHEN TO RECOGNIZE IT

When you compare a string from both ends while ignoring non-essential characters.

VISUAL HOOK

A , m ... a

1. Move left/right inward.
2. Skip non-alphanumeric chars first.
3. Compare lowercase real chars, then move both.

Two pointers walk inward. Each pointer skips noise until both point at real comparable characters.

CORE MOVE

Move left and right inward. If a side is not alphanumeric, skip it. Once both are valid, compare lowercase characters.

CODE SKELETON

```
l = 0; r = s.length - 1
while l < r:
  if !isAlphaNum(s[l]): l++; continue
  if !isAlphaNum(s[r]): r--; continue
  if lower(s[l]) != lower(s[r]): return false
  l++; r--
return true
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Do the skip before compare.
- Only move both pointers after you have compared two valid characters.

#05 ValidAnagram

Easy

Frequency balance Easy Source LeetCode

Before reading: why does order not matter here?

PROBLEM DESCRIPTION

Given two strings, check whether they contain exactly the same characters with exactly the same counts.

INPUT

```
s = "listen"
t = "silent"
```

OUTPUT

```
true
```

TIME	SPACE	DATA STRUCTURES
O(n)	O(1)	int[26] frequency array

WHEN TO RECOGNIZE IT

When order does not matter, but exact character counts do.

VISUAL HOOK

l:+1 i:+1 s:+1

s:-1 i:-1 l:-1

1. Reject if lengths differ.
2. Add counts from first string.
3. Subtract counts from second string; all counts must end zero.

Use a balance scale per letter: first string adds weight, second string removes it. All scales must end at zero.

CORE MOVE

Check lengths first. Increment counts for s, decrement counts for t, then verify every count is zero.

CODE SKELETON

```
if s.length != t.length: return false
count = int[26]
for c in s: count[c-'a']++
for c in t: count[c-'a']--
return every count == 0
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Array size 26 assumes lowercase English letters.
- For Unicode or mixed characters, use a map instead.

#06 ShortestWordDistance

Easy

Last seen positions Easy Source LeetCode

Before reading: what are the latest two pins you need to remember?

PROBLEM DESCRIPTION

Given a list of words and two target words, return the smallest distance between any occurrence of the two targets.

INPUT

```
words =
["the","quick","brown","fox","jumps","over","
the","lazy","dog"]
word1 = "fox", word2 = "dog"
```

OUTPUT

```
5
```

TIME
O(n)

SPACE
O(1)

DATA STRUCTURES
Last seen indexes

WHEN TO RECOGNIZE IT

When distance depends only on the latest seen positions of two target words.

VISUAL HOOK

1. Scan words once.
2. Update latest index for word1 or word2.
3. When both are known, update minimum gap.

Drop pins on the latest word1 and word2 positions. Every new pin gives a fresh distance candidate.

CORE MOVE

During one scan, update idx1 or idx2 when you see either word. Once both exist, update the minimum absolute difference.

CODE SKELETON

```
idx1 = idx2 = -1; best = INF
for i in range(words):
  if words[i] == word1: idx1 = i
  if words[i] == word2: idx2 = i
  if idx1 != -1 && idx2 != -1: best =
min(best, abs(idx1-idx2))
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Single pass is simpler than storing every index.
- If word1 and word2 can be the same word, that is a different variant.

#07 GoodPair

Easy

Frequency creates pairs Easy Source LeetCode

Before reading: when a duplicate arrives, how many new pairs does it create?

PROBLEM DESCRIPTION

Given an integer array, count pairs of indices i and j where i is before j and both positions contain the same value.

INPUT

```
nums = [1, 2, 3, 1, 1, 3]
```

OUTPUT

```
4
```

TIME
O(n)

SPACE
O(k)

DATA STRUCTURES
Frequency HashMap

WHEN TO RECOGNIZE IT

When every new duplicate can pair with all earlier copies of the same number.

VISUAL HOOK

1	2	3	1	1	3

1. Keep frequency of values already seen.
2. A new value forms frequency[value] pairs.
3. Then increment its frequency.

Think handshake count: the new 1 shakes hands with every previous 1.

CORE MOVE

Keep a frequency map. For each number, add its current frequency to the answer, then increment that frequency.

CODE SKELETON

```
pairs = 0
freq = new HashMap()
for n in nums:
  pairs += freq.getOrDefault(n, 0)
  freq[n]++
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Add before incrementing, because the current item only pairs with previous items.
- This is easier than computing nC2 at the end.

#08 Sqrt

Medium

Binary answer search Medium Source LeetCode

Before reading: what does mid squared tell you to discard?

PROBLEM DESCRIPTION

Given a non-negative integer x, return the floor of its square root without using a built-in square-root operation.

INPUT

`x = 2147395600`

OUTPUT

`46340`

TIME
O(log x)

SPACE
O(1)

DATA STRUCTURES
Binary search bounds

WHEN TO RECOGNIZE IT

When the answer is a number in a range and you can test whether a guess is too small or too large.

VISUAL HOOK

1. Search possible roots, not array indexes.
2. mid^2 too small means answer is to the right.
3. mid^2 too large means answer is to the left; return right after crossing.

Put mid on the number line. mid squared tells which half of possible roots can be discarded.

CORE MOVE

Binary search candidate roots. If mid*mid is less than x, move left up. If greater, move right down. Return right after crossing.

CODE SKELETON

```
left = 0; right = x / 2
while left <= right:
  mid = left + (right-left)/2
  square = (long) mid * mid
  if square < x: left = mid + 1
  else if square > x: right = mid - 1
  else return mid
return right
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Use long for mid * mid to avoid overflow.
- Return right at the end because left has moved to the first too-large value.

#09 ClosestToZero

Easy

Best-so-far scan Easy Source LeetCode

Before reading: what value is closest on the number line, and what happens on a tie?

PROBLEM DESCRIPTION

Given an integer array, return the number closest to zero. If negative and positive values are equally close, return the positive one.

INPUT

```
nums = [2, -1, 1]
```

OUTPUT

```
1
```

TIME
O(n)

SPACE
O(1)

DATA STRUCTURES
Best-so-far variable

WHEN TO RECOGNIZE IT

When the answer is one best element and each number can be judged independently.

VISUAL HOOK

1. Track one best number.
2. Compare absolute distance to zero.
3. If tied, keep the larger value.

Draw a number line around zero. Distance decides first; if distance ties, choose the right-side positive number.

CORE MOVE

Track the current closest number. Replace it when absolute value is smaller, or when absolute value ties and the new value is larger.

CODE SKELETON

```
best = nums[0]
for n in nums:
  if abs(n) < abs(best): best = n
  else if abs(n) == abs(best): best =
max(best, n)
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Tie rule is the entire trick: prefer +x over -x.
- Initialize from the array if possible; sentinel values can make tie logic feel weird.

#10 MergeStringsAlternately

Easy

Alternating append Easy Source LeetCode

Before reading: what happens after one string runs out?

PROBLEM DESCRIPTION

Given two strings, create a new string by alternating characters from each one, then append any leftover tail from the longer string.

INPUT

```
word1 = "abcd"
word2 = "pq"
```

OUTPUT

```
apbqcd
```

TIME
O(n + m)

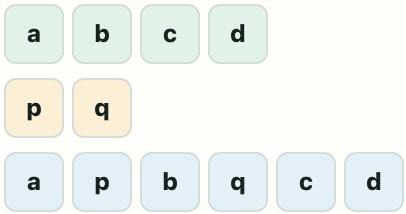
SPACE
O(n + m)

DATA STRUCTURES
StringBuilder, two indexes

WHEN TO RECOGNIZE IT

When the output is formed by weaving two strings character by character.

VISUAL HOOK



1. Append one char from word1.
2. Append one char from word2.
3. Append the leftover suffix from the longer string.

Zip the two rows together: top, bottom, top, bottom; then drag the leftover tail to the end.

CORE MOVE

Use two indexes and a StringBuilder. Append word1[i], word2[j] while both are in range, then append remaining substrings.

CODE SKELETON

```
while i < word1.length && j < word2.length:
  sb.append(word1[i++])
  sb.append(word2[j++])
sb.append(word1.substring(i))
sb.append(word2.substring(j))
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Do not forget leftovers from the longer string.
- StringBuilder avoids repeated string concatenation.

#11 CheckIfStringsEquivalent

Easy

Chunked strings Easy Source LeetCode

Before reading: what final string does each array form?

PROBLEM DESCRIPTION

Given two arrays of string chunks, decide whether joining all chunks in each array creates the same final string.

INPUT

```
word1 = ["ab", "c"]
word2 = ["a", "bc"]
```

OUTPUT

```
true
```

TIME
O(total chars)

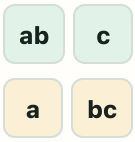
SPACE
O(total chars)

DATA STRUCTURES
StringBuilder x 2

WHEN TO RECOGNIZE IT

When two string arrays are really just two broken-up versions of a final string.

VISUAL HOOK



1. Flatten word1 chunks in order.
2. Flatten word2 chunks in order.
3. Compare the final strings.

Mentally flatten both rows into one line: [ab][c] -> abc and [a][bc] -> abc.

CORE MOVE

Append all pieces of word1 into one StringBuilder, append all pieces of word2 into another, then compare the final text.

CODE SKELETON

```
StringBuilder a, b
for part in word1: a.append(part)
for part in word2: b.append(part)
return a.toString().equals(b.toString())
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Use content comparison, not reference comparison.
- If this becomes huge, compare character streams instead of building both full strings.

#12 IsSubsequence

Easy

Order check Easy Source LeetCode

Before reading: which pointer only moves on a successful match?

PROBLEM DESCRIPTION

Given strings s and t, check whether s can be formed by deleting characters from t without changing the order of the remaining characters.

INPUT

```
s = "abc"
t = "ahbgdc"
```

OUTPUT

```
true
```

TIME
O(|t|)

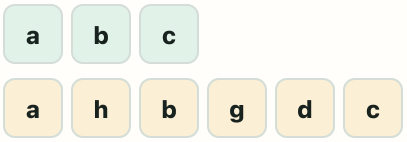
SPACE
O(1)

DATA STRUCTURES
Two pointers

WHEN TO RECOGNIZE IT

When one string only needs to appear in order inside another string, not contiguously.

VISUAL HOOK



1. Keep one pointer on s.
2. Sweep t from left to right.
3. Advance s only when current needed char appears.

Keep a finger on s. Sweep across t and move the s finger only when the current needed char appears.

CORE MOVE

Scan t with one pointer. When t matches the current char of s, advance the s pointer. Success means all of s was consumed.

CODE SKELETON

```
i = 0
for ch in t:
  if i < s.length && s.charAt(i) == ch:
    i++
return i == s.length
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- Empty s should return true.
- Guard `s.charAt(i)` when i reaches s.length(); otherwise empty or fully matched s can crash.

#13 MoveZeroes

Easy

Stable compaction Easy Source LeetCode

Before reading: where is the next non-zero allowed to be written?

PROBLEM DESCRIPTION

Given an integer array, move all zeroes to the end while keeping the non-zero values in their original relative order.

INPUT

`nums = [0, 1, 3, 0, 12]`

OUTPUT

`[1, 3, 12, 0, 0]`

TIME
O(n)

SPACE
O(1)

DATA STRUCTURES
Write pointer

WHEN TO RECOGNIZE IT

When unwanted values move to the end but the wanted values must keep their order.

VISUAL HOOK

1. write points to next non-zero slot.
2. Scan all numbers and copy non-zero values forward.
3. Fill everything from write to end with zero.

Imagine a write slot moving forward only when a non-zero is copied into it. Everything behind write is compacted.

CORE MOVE

Use a write pointer for the next non-zero slot. Scan all values, copy non-zero values to write, then fill the rest with zero.

CODE SKELETON

```
write = 0
for n in nums:
  if n != 0: nums[write++] = n
while write < nums.length:
  nums[write++] = 0
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- This is not a sorting problem.
- The generic template should scan from index 0 so the first element is handled cleanly.

#14 FlippingImage

Easy

Mirror and invert Easy Source LeetCode

Before reading: how can one left/right pass both reverse and invert a row?

PROBLEM DESCRIPTION

Given a binary matrix, flip each row horizontally, then invert every bit so 0 becomes 1 and 1 becomes 0.

INPUT

`image = [[1,1,0,0], [1,0,0,1], [0,1,1,1], [1,0,1,0]]`

OUTPUT

`[[1,1,0,0], [0,1,1,0], [0,0,0,1], [1,0,1,0]]`

TIME
O(rows * cols)

SPACE
O(1)

DATA STRUCTURES
In-place matrix, two pointers

WHEN TO RECOGNIZE IT

When each row must be mirrored and each binary value must be toggled.

VISUAL HOOK

1	1	0	0
1	1	0	0

1. For each row, put pointers at both ends.
2. Swap mirrored cells while writing inverted values.
3. When pointers meet, invert the middle too.

Treat every row like a mirror. Left receives inverted right; right receives inverted old left.

CORE MOVE

For each row, keep left and right pointers. Put invert(right) on the left and invert(old left) on the right.

CODE SKELETON

```
for row in image:
  l = 0; r = row.length - 1
  while l <= r:
    temp = row[l]
    row[l] = invert(row[r])
    row[r] = invert(temp)
    l++; r--
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- When left meets right, still invert the middle cell.
- Use a saved temp before overwriting the left value.

#15 MajorityElement

Easy

Vote cancellation Easy Source LeetCode

Before reading: how does the majority survive cancellation?

PROBLEM DESCRIPTION

Given an array where one value appears more than half the time, return that majority value.

INPUT

```
nums = [2, 2, 1, 1, 1, 2, 2]
```

OUTPUT

```
2
```

TIME	SPACE	DATA STRUCTURES
O(n)	O(1)	Candidate + vote count

WHEN TO RECOGNIZE IT

When the problem guarantees one value appears more than half the array.

VISUAL HOOK

1. Hold a candidate and vote count.
2. Same value adds a vote.
3. Different value cancels a vote; reset when count hits zero.

Picture values fighting in pairs. Different values cancel; the true majority has extra copies left.

CORE MOVE

Keep a candidate and count. Same value increments count, different value decrements. When count hits zero, choose a new candidate.

CODE SKELETON

```
candidate = nums[0]; count = 0
for n in nums:
  if count == 0: candidate = n
  count += (n == candidate) ? 1 : -1
return candidate
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- This works without verification only when the majority is guaranteed.
- The idea is cancellation, not counting exact frequencies.

#16 FindNumbersWithEvenDigits

Easy

Digit count Easy Source LeetCode

Before reading: are you solving with number value or digit length?

PROBLEM DESCRIPTION

Given an array of integers, count how many numbers have an even number of decimal digits.

INPUT

```
nums = [12, 345, 2, 6, 7896]
```

OUTPUT

```
2
```

TIME
O(total digits)

SPACE
O(1)

DATA STRUCTURES
Digit counter

WHEN TO RECOGNIZE IT

When the value itself does not matter, only the length of its decimal representation.

VISUAL HOOK

`12` `345` `2` `6` `7896`

1. Convert each number to a digit count.
2. Check count parity.
3. Increment answer for even counts.

Read each number as text blocks: 12 has two blocks, 345 has three, 7896 has four.

CORE MOVE

Turn each number into text and count its length, or divide by 10 repeatedly. Add one when the count is even.

CODE SKELETON

```
ans = 0
for num in nums:
  digits = String.valueOf(num).length()
  if digits % 2 == 0: ans++
```

BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.

BUG MAGNETS

- If using math, remember 0 has one digit.
- String conversion is perfectly fine for this warmup problem.